
Development of a Method for Detecting Attacks on Mobile Banking Applications Using Dynamic Analysis

By

IGOR ZAITSEV



School of Cybersecurity
ASTANA IT UNIVERSITY

6B06301 - Cybersecurity
Supervisor: Akzhibek Amirova, Senior Lecturer

JUNE 2026
ASTANA

Abstract

This thesis addresses a runtime security gap in Android mobile banking: several relevant attack behaviors become observable only while the application is executing. Programmatic touch events enter the Android input pipeline, overlay activity modifies window state, and C2 exfiltration becomes visible when the host process constructs a network destination. These behaviors cannot be reconstructed from the APK alone and external network monitoring sees only part of the picture.

A Frida-based research prototype, *RASP Dynamic Analyzer v2.0*, was implemented for the intentionally vulnerable InsecureBankv2 application. The prototype installs five interception points and six deterministic detection rules: `View.dispatchTouchEvent` for ADB touch injection (Hook #1) and for non-focusable overlays via the `MotionEvent` obscured-window flags (Hook #2b); `onWindowFocusChanged` with an IME filter for focus-stealing overlays (Hook #2); `DefaultHttpClient.execute`, a guarded `okhttp3` extension and `java.net.URL.$init` for network destination construction (Hooks #3a, #3a-ext, #3b). The design records an interpretable reason for every CRITICAL event rather than using ML classification.

Evaluation was performed on an Android API 35 emulator under a repeated-trial protocol. After the report-capture harness was stabilized, all three modeled attack vectors were detected in 20/20 valid runs, yielding a detection rate of 100%. A dedicated controlled-overlay APK experiment reported 20/20 batch-level detections, with 17 of 18 retained per-run audit reports strictly satisfying all four verification criteria. A benign baseline of four legitimate user scenarios produced zero CRITICAL false positives. The work does not claim complete coverage of Android banking malware; it demonstrates that selected runtime hooks produce auditable evidence for specific attack behaviors, supported by a literature review of 30 selected academic and official sources.

Keywords: RASP, dynamic analysis, Frida, mobile banking, ATS, overlay, C2 exfiltration, Android, runtime hooks, behavioral analysis

Dedication and Acknowledgements

I would like to express my gratitude to my scientific supervisor, Akzhibek Amirova, for her guidance throughout the development of this thesis and for her patience during the many iterations of the prototype design. I am also thankful to the staff of the School of Cybersecurity at Astana IT University for providing a constructive academic environment.

Special thanks go to my family for their continuous support during the months of research and implementation work.

Author's Declaration

I declare that the work in this diploma thesis was carried out in accordance with the requirements of the University's Regulations and that it has not been submitted for any other academic award. Except where indicated by specific references in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others is indicated as such. Any views expressed in the dissertation are those of the author.

SIGNED:

DATE:

Definitions

Following terms are used in this work:

ATS	Automated Touch Simulation - the class of attacks in which a program injects synthetic touch events into a target application without physical user interaction.
Overlay attack	A class of UI deception attacks in which a malicious application draws a window on top of a legitimate application to intercept user input or mislead the user.
C2 exfiltration	Command-and-Control exfiltration - the unauthorized transmission of sensitive data from an infected application or device to a remote server controlled by an attacker.
RASP	Runtime Application Self-Protection - a class of defense techniques in which protective logic operates inside the running application process rather than as an external monitor.
Dynamic instrumentation	The technique of injecting auxiliary code into a running program in order to observe or modify its behavior without recompilation of the original binary.
Hook	In the context of this thesis, an instrumentation point at which a function call is intercepted by the agent before being delegated to the original implementation.
Frida agent	A JavaScript program loaded by the Frida framework into the address space of a target process for the purpose of dynamic instrumentation.

IME	Input Method Editor - the Android subsystem that handles soft keyboards and other text input mechanisms.
deviceId	The integer identifier returned by <code>MotionEvent.getId()</code> on Android, used to identify the input device that generated an event.

Designations and Abbreviations

Following designations and abbreviations are used in this work:

ADB	Android Debug Bridge
AITU	Astana IT University
API	Application Programming Interface
APK	Android Package
ATS	Automated Touch Simulation
AVD	Android Virtual Device
C2	Command and Control
CEF	Common Event Format
CSV	Comma-Separated Values
DR	Detection Rate
ENISA	European Union Agency for Cybersecurity
FN	False Negative
FP	False Positive
FPR	False Positive Rate
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
IME	Input Method Editor
IPC	Inter-Process Communication
JSON	JavaScript Object Notation

JVM	Java Virtual Machine
ML	Machine Learning
MTD	Mobile Threat Defense
NIST	National Institute of Standards and Technology
OS	Operating System
OWASP	Open Web Application Security Project
RASP	Runtime Application Self-Protection
RPC	Remote Procedure Call
SIEM	Security Information and Event Management
SOC	Security Operations Center
TN	True Negative
TP	True Positive
TTD	Time to Detect
UI	User Interface
URL	Uniform Resource Locator

List of Tables

TABLE	Page
1.1 Comparison of defense methods for the three attack vectors considered in this thesis	5
2.1 Research testbed configuration	11
2.2 Detection decision rules per hook	11
2.3 Design decisions and alternatives considered	13
3.1 Data collection methodology	21
3.2 Simulated attack scenario parameters	23
3.3 Repeated-trial detection results after harness stabilization (20 attempted runs, 20 valid)	24
3.4 End-to-end latency for the reference session (run 20), measured from hook activation	25
3.5 Controlled overlay APK scenario - repeated-trial detection results	26
3.6 Benign baseline results (four legitimate scenarios)	28
3.7 Confusion matrix per attack vector (20 valid attack runs, 4 benign actions) .	29
3.8 Evaluation metrics summary	30

List of Figures

FIGURE	Page
2.1 Architecture of <i>RASP Dynamic Analyzer v2.0</i> and the data flow between its components	10
2.2 agent.js entry point	13
2.3 ATS detection heuristic in Hook #1	14
2.4 Overlay-proxy detection with IME filter in Hook #2	14
2.5 Real overlay detection through obscured-touch flags in Hook #2b	15
2.6 TRUSTED_HOSTS whitelist and classification logic	15
3.1 Runtime analyzer output during the combined attack scenario	24
3.2 C2 server log line (port 9999)	29
3.3 JSON audit report structure	30
1 Essential runtime hooks in <code>agent.js</code>	44
2 Essential <code>OverlayTestApp</code> window parameters	45
3 Minimal simulator logic for overlay and C2 stimuli	46
4 Essential event counting rule used by the batch runner	46
5 Compact JSON audit schema example	47

Table of Contents

Abstract	i
Dedication and Acknowledgements	ii
Author’s Declaration	iii
Definitions	iv
Designations and Abbreviations	vi
List of Tables	viii
List of Figures	ix
Introduction	xiii
1 Literature Review and Background	1
1.1 Mobile Banking Threat Landscape	1
1.1.1 Statistical Context and Relevance	1
1.1.2 Automated Touch Simulation (ATS)	2
1.1.3 Overlay Attacks	2
1.1.4 C2 Network Exfiltration	3
1.2 Existing Defense Methods and Their Limitations	3
1.2.1 Static Analysis	4
1.2.2 External Network Monitoring	4
1.2.3 ML-Based Classifiers	4
1.2.4 Dynamic Instrumentation Approaches	5
1.2.5 Comparative Summary	5

1.3	The Frida Dynamic Instrumentation Platform	5
1.4	InsecureBankv2 as a Research Target	6
1.5	Summary of Chapter 1	6
2	System Design and Implementation	8
2.1	Threat Model and Assumptions	8
2.1.1	Threat Model	8
2.1.2	Assumptions	9
2.2	System Architecture	10
2.2.1	Overview	10
2.2.2	Testbed Configuration	10
2.3	Detection Logic and Decision Rules	11
2.4	Design Decisions and Alternatives Considered	12
2.5	Component Implementation	13
2.5.1	agent.js: the RASP Agent	13
2.5.2	analyzer.py: the Controller	16
2.5.3	simulate_attack.py: the Attack Simulator	16
2.5.4	server.py: the Dual-Port Mock Server	16
2.5.5	OverlayTestApp: the Controlled Overlay Stimulus	17
2.5.6	batch_run.py and benign_baseline.py: the Evaluation Scripts . .	17
2.6	Summary of Chapter 2	18
3	Experiment and Evaluation	20
3.1	Methodology of Data Collection	20
3.1.1	Data Sources and Collection Methods	20
3.1.2	Justification of Data Sources	21
3.1.3	Justification of InsecureBankv2 as Research Target	21
3.2	Experimental Procedure	22
3.2.1	Single-Run Procedure	22
3.2.2	Repeated-Trial Protocol	22
3.2.3	Benign Baseline	23
3.2.4	Attack Scenario Parameters	23
3.3	Experimental Results	23
3.3.1	Hook Initialization	23
3.3.2	Repeated-Trial Detection Results	24

3.3.3	Single-Run Detail and Latency	25
3.3.4	Harness Stabilization and Report-Capture Verification	25
3.3.5	Controlled Overlay APK Scenario	26
3.3.6	Benign Baseline Results	27
3.3.7	C2 Traffic Verification	28
3.4	Classification Analysis	29
3.4.1	Confusion Matrix	29
3.4.2	Evaluation Metrics	30
3.4.3	JSON Audit Report	30
3.5	Interpretation of Results	31
3.5.1	ATS Detection	31
3.5.2	Overlay Proxy Detection	31
3.5.3	C2 Exfiltration Detection	32
3.5.4	Reliability after Harness Stabilization	32
3.6	Limitations and Threats to Validity	32
3.6.1	Internal Validity	32
3.6.2	External Validity	33
3.6.3	Threats to Validity	33
3.7	Future Work	34
3.8	Summary of Chapter 3	34
Conclusion		36
Bibliography		39
Appendix A		44
	Runtime Detection Rules in <code>agent.js</code>	44
	Controlled Overlay and C2 Stimuli	45
	Report Capture and Counting Rule	46

Introduction

Background and motivation. This thesis examines Android mobile banking security from a runtime-observation perspective. Some attack actions become visible only while the application is executing: programmatic touches pass through the Android input pipeline, overlay activity changes window state, and C2 exfiltration appears when the host process constructs a network destination. These signals cannot be reliably reconstructed from the APK alone, while external network monitoring sees only part of the context [1-7].

Problem statement. The three selected behaviors occur at different Android runtime layers: ATS in the input event path, overlay behavior in focus transitions or obscured-touch flags, and C2 activity during URL or HTTP destination construction. Static analysis is useful before execution, but it cannot see externally generated input and focus events. External traffic monitoring loses application-level context, and ML classifiers often do not provide event-level explanations for audit and SOC use [8-10]. Therefore, this work focuses on in-process runtime observation through dynamic instrumentation.

Research goal. To develop and experimentally evaluate a method for detecting runtime attacks on Android mobile banking applications using dynamic behavioral analysis through the Frida instrumentation platform.

Research tasks. The work includes a literature review of academic and official sources, comparison of static APK analysis, external traffic monitoring, ML-based detection and runtime instrumentation, design of a Frida-based RASP prototype, implementation of a JavaScript agent, Python controller, ADB scenario runner, dual-port mock server, JSON audit generation, repeated-trial batch runner and benign-baseline tester, and evaluation of attack and legitimate-user scenarios on Android API 35.

Object and subject of research. The object is the execution of InsecureBankv2 in a controlled Android API 35 emulator environment. The subject is the use of deterministic runtime heuristics to classify selected security-relevant events: ADB-based touch injection, suspicious focus loss, controlled overlay events through obscured-touch flags and non-whitelisted network destination construction.

Hypothesis and methods. The hypothesis is that ATS bot activity, overlay-like behavior and C2-like exfiltration can be detected without modifying the target application source code and without CRITICAL false positives in the tested benign scenarios. The work uses experimental, engineering and analytical methods: controlled stimulus reproduction, prototype implementation, event interpretation, threat taxonomy and metrics calculation.

Scientific novelty and practical significance. The contribution is not a universal malware classifier, but a reproducible runtime attack behavior detection workflow. The prototype combines five interception points and six deterministic rules, preserves human-readable reasons in JSON audit events and can be developed further through Frida Gadget, native integration or Mobile Threat Defense reference implementation.

Scope and structure. The work is a research prototype tested on one intentionally vulnerable application and emulator environment. Chapter 1 presents the literature review, Chapter 2 describes system design and implementation, Chapter 3 presents methodology, results, limitations and future work, and Appendix A contains only essential implementation excerpts.

Chapter 1

Literature Review and Background

This chapter provides the literature foundation for the rest of the thesis. It is organised around three goals. First, to position mobile banking threats with respect to documented industry and academic evidence. Second, to explain why the three behaviors selected for this work - ATS injection, overlay-related window-focus changes and C2-like exfiltration - require runtime visibility. Third, to compare defense methods that have been proposed in the literature and to motivate the dynamic instrumentation approach adopted in the prototype.

1.1 Mobile Banking Threat Landscape

1.1.1 Statistical Context and Relevance

For this thesis the relevance of mobile banking security is considered through the behaviors that can be observed during an active Android session. Industry telemetry shows that mobile financial threats remain a persistent component of the cybercrime ecosystem [1], while ENISA continues to list malware, account compromise, social engineering and data breaches among the leading risks for the European financial sector [2]. These reports justify the domain but do not by themselves define the technical detection point used in this work.

The technical motivation arises from Android malware and mobile banking studies. Tam et al. [18] describe the evolution of Android malware and the role of both static and dynamic analysis. Chen et al. [19] show that mobile banking applications continue to expose sensitive data through implementation weaknesses that are visible at runtime.

Faruki et al. [20] provide an earlier survey of issues, malware penetration and defenses on Android. Tsobdjou et al. [21] present a recent security-assessment framework specifically for Android mobile banking applications, supporting the domain relevance of the present work. In the thesis these broader findings are narrowed to three runtime behaviors that can be reproduced on a controlled testbed: programmatic touch input, suspicious focus loss and construction of an untrusted network destination.

1.1.2 Automated Touch Simulation (ATS)

ATS attacks involve the programmatic generation of touch events to simulate legitimate user interaction. Implementations include `adb input tap` commands, abuse of Android Accessibility Services and the use of UI automation frameworks [11, 22].

Each Android touch event is encapsulated in a `MotionEvent` object. The `getDeviceId()` method returns an integer that identifies the input device that generated the event [17]. Physical devices - the touchscreen, an external mouse, a stylus - receive non-negative identifiers assigned by the operating system at device registration. Events injected via `adb input tap` have no associated hardware device and consistently receive `deviceId = -1` in the tested environment, which is an Android API 35 emulator with the Google APIs system image. This attribute is used as the ATS detection heuristic in the developed prototype.

Scope limitation. ATS attacks implemented via Android Accessibility Services may produce different `deviceId` values depending on the Android version and the device. The `deviceId < 0` heuristic was validated only for the `adb input tap` simulation method used in this study.

1.1.3 Overlay Attacks

Overlay attacks involve drawing a transparent or opaque window over the user interface of a legitimate banking application in order to intercept user credentials [3]. Fratantonio et al. [4] documented the “Cloak and Dagger” attack family, demonstrating that combinations of the `SYSTEM_ALERT_WINDOW` and Accessibility permissions enable persistent overlay-based attacks against the Android UI feedback loop. Yan et al. [5] performed a market-scale study of overlay-based Android malware and showed that overlay-related behavior can be used as a practical signal for detecting malicious applications. These findings justify the use of window-focus and overlay-related indicators in mobile threat analysis.

Scope limitation. In the initial version of this work the Overlay detector was validated through a controlled focus-loss proxy triggered by `am start android.settings.SETTINGS`. In the updated version, a controlled `OverlayTestApp` was added to exercise a real `SYSTEM_ALERT_WINDOW / TYPE_APPLICATION_OVERLAY` condition; it is detected by a complementary rule (Hook #2b) that inspects the `MotionEvent` obscured-window flags. This remains a benign controlled stimulus and does not represent the full diversity of real-world overlay malware.

1.1.4 C2 Network Exfiltration

Command-and-Control (C2) exfiltration occurs when malicious code embedded within (or injected into) a banking application process silently transmits sensitive data to an attacker-controlled server [1, 23]. For the present thesis the relevant problem is not only that traffic exists but that the decision point appears before or during runtime construction of a network destination. Prior Android dynamic-analysis systems such as TaintDroid and DroidScope demonstrate why in-process observation can expose information flows and runtime behavior that are unavailable to APK-only inspection [6, 7].

Scope limitation. In this work C2 exfiltration is simulated via a controlled Frida-triggered construction of a `java.net.URL` inside the process, rather than by a real injected malware component. The simulation validates that the `URL.$init` hook can intercept such activity before network transmission.

1.2 Existing Defense Methods and Their Limitations

The reviewed defense methods are compared by their observation point. The three behaviors modeled in this thesis do not appear at a single layer of the system: touch injection is visible in Android input events; overlay-related behavior is visible in focus changes; and C2-like activity is visible when the host process prepares a network destination. The following subsections therefore discuss not only whether a method is useful in general, but whether it can observe the concrete event attributes used by the prototype for classification.

1.2.1 Static Analysis

In this thesis static analysis is treated as a baseline rather than as the principal detection mechanism. Static APK inspection is valuable for identifying suspicious permissions, insecure API usage, taint-flow problems and known malware patterns [14, 20, 23, 24]. The practical limitation is that the ATS and Overlay-proxy events used in the experiment are not stored inside the InsecureBankv2 APK: they are generated after the application is already running, either through `adb input tap` or through a focus-changing Android activity. Static inspection also cannot observe the runtime `MotionEvent` object or the IME state that the proposed method uses for classification. Static analysis is therefore used here only as background; the detector itself is moved into the running process. Alzaylaee et al. [25] similarly demonstrate that observed runtime behavior depends substantially on how the application is exercised during analysis.

1.2.2 External Network Monitoring

External network monitoring observes the communication channel after the application has already created a connection. This makes it useful for traffic-level visibility but insufficient for two of the three scenarios selected here: ATS and Overlay-proxy events do not require network traffic at all. Even in the C2-like case an external observer does not know which Java call constructed the destination, whether it occurred during a login flow or which in-process rule considered it suspicious. The prototype therefore follows the runtime-observation line of prior Android analysis work [6, 7, 26] by logging the destination from inside the process before the connection is completed.

1.2.3 ML-Based Classifiers

ML-based Android malware classifiers are useful when the task is to classify applications or samples at scale [9, 10, 18]. The task in the present thesis is narrower: the prototype must explain why one runtime event during a banking session is suspicious. For that reason the implemented method uses deterministic rules rather than a trained model: `deviceId < 0` for the selected ADB-based ATS scenario, focus loss without visible IME for the Overlay-proxy scenario and a non-whitelisted destination for C2-like exfiltration. These rules are less flexible than ML classifiers but their decisions can be read directly in the JSON audit. This design choice follows the interpretability requirement discussed by Singh et al. [8].

1.2.4 Dynamic Instrumentation Approaches

Dynamic analysis has a substantial research history in Android security. Crowdroid used behavior-based system-call traces for Android malware detection [26]. DroidScope reconstructed the OS-level and Dalvik-level semantic views for malware analysis [7]. AppsPlayground automated dynamic security analysis of smartphone applications [27]. Irolla and Filiol [28] demonstrated that dynamic analysis on real devices can execute a substantially larger fraction of application code than UI-fuzzing tools alone. Enck et al. [6] demonstrated in-process taint tracking with TaintDroid as a related approach for monitoring sensitive information flows. These works collectively establish that runtime visibility is a viable foundation for Android security research, even when the specific instrumentation backend differs.

1.2.5 Comparative Summary

Table 1.1 summarizes the detection capabilities and limitations of the reviewed approaches for the three scenarios considered in this thesis.

Table 1.1: Comparison of defense methods for the three attack vectors considered in this thesis

Method	ATS	Overlay	C2	Interp.	Key limitation
Static analysis	No	No	Partial	Yes	Misses runtime behavior; APK has no MotionEvent object
Network monitoring	No	No	Partial	Yes	Blind to non-network events; HTTPS opaque externally
ML classifiers	Yes	Yes	Yes	Low	No event-level explanation; training data required
Dynamic instrumentation	Yes	Yes	Yes	Yes	Requires privileged access (frida-server needs root)
This work	Yes	Focus-loss proxy + controlled overlay APK	Yes	Yes	Emulator-only; single app; controlled stimuli; partial ATS coverage

1.3 The Frida Dynamic Instrumentation Platform

Frida [16] is an open-source cross-platform dynamic instrumentation toolkit that supports Android, iOS, Windows, macOS and Linux. It enables the injection of JavaScript agents into running processes and provides access to the Java Virtual Machine through the Java bridge object. Hooks are implemented by replacing function pointers in the Java

bytecode layer, which makes interception of Java methods possible without modification of the application source code.

The relevant Frida components for this work are:

- **frida-server**: a daemon running on the target device with elevated privileges, accepting instrumentation commands from the host;
- **JavaScript Bridge (Java object)**: provides access to Android’s Java runtime, enabling class loading, method overriding and object manipulation;
- **frida-tools / frida CLI**: command-line utilities for managing instrumentation sessions and loading script files.

A specific limitation identified during this work is that in Frida 17.x the Java Bridge does not initialize correctly when the host connects via the Python API to frida-server over USB. The Frida CLI, in contrast, initializes the bridge reliably. This required launching Frida via `subprocess.Popen` from the Python controller rather than using the `frida` Python library directly.

1.4 InsecureBankv2 as a Research Target

InsecureBankv2 [29] was selected because it provides the experiment with a banking-style workflow without involving a real financial application. The application has a login screen, a configurable backend endpoint and HTTP communication that can be reproduced on a local mock server. These properties are sufficient for testing whether the prototype can observe input events, focus changes and process-level URL construction. The application is intentionally vulnerable and educational and is therefore used here as a controlled research target rather than as evidence that the same results would transfer directly to production banking software.

1.5 Summary of Chapter 1

The literature review supports the following positioning for the remainder of the thesis. The selected attack behaviors require visibility inside the running Android application or close to its UI event flow. Static analysis remains useful for pre-execution inspection, and

ML remains useful for sample classification, but neither approach directly produces the event-level evidence required by this prototype. Dynamic instrumentation is therefore adopted as the practical observation mechanism, and deterministic rules are used to keep each alert explainable for later audit and SOC-style interpretation [8]. Recent mobile-banking security assessment work [21] supports the relevance of controlled Android banking evaluation, provided that the limitations of the testbed are stated explicitly.

Three design implications follow from the review and shape the next chapter. First, the prototype must observe events inside the running application rather than from outside its process boundary, because the selected behaviors (ATS, focus-loss overlay activity, in-process URL construction) are only visible at runtime. Second, the rule set must remain interpretable at the event level so that each generated alert can be tied to a specific runtime attribute - the `deviceId` of a `MotionEvent`, the IME visibility state, the presence of the obscured-window flag, or the host portion of a constructed URL - rather than to an opaque classifier score. Third, the evaluation must use a repeated-trial protocol with an explicit benign baseline so that the resulting numbers can be reported as detection rate and false-positive rate rather than as a single anecdotal measurement.

Chapter 2 acts on these three implications. It defines the threat model and assumptions, the interception points and detection rules implemented in the Frida agent, the controller and scenario runner, the dual-port mock server, the controlled overlay APK `OverlayTestApp` used to exercise the obscured-touch rule, and the two evaluation scripts (batch runner and benign-baseline tester) that produce the repeated-trial statistics reported in Chapter 3.

Chapter 2

System Design and Implementation

This chapter describes the design and implementation of the RASP Dynamic Analyzer v2.0 prototype. It begins with the threat model and the assumptions adopted for the experiment, continues with the system architecture and detection rules, and concludes with the implementation details of each software component.

2.1 Threat Model and Assumptions

2.1.1 Threat Model

The threat model considers an Android mobile banking application running on a user’s device. The capabilities and goals of the adversary for each modelled vector are stated below.

Vector 1 (ATS). The attacker can send programmatic touch events to the device through ADB or accessibility mechanisms. The goal is to simulate user actions, such as PIN brute-forcing or unattended confirmation of transactions, without physical interaction.

Vector 2 (Overlay). The threat model covers two complementary stimuli for overlay-like behavior. The first is a focus-loss *proxy*: a system event causes the banking application to lose window focus, which is the signal exploited by overlays that capture focus. The second is a *controlled real overlay APK* (`OverlayTestApp`, package `kz.aitu.overlaytest`) built specifically for this evaluation. It uses the `SYSTEM_ALERT_WINDOW` permission and draws a `TYPE_APPLICATION_OVERLAY` window with the fixed text “TEST OVERLAY

ACTIVE” on top of the banking UI. The overlay is deliberately benign: it does not capture credentials, it never receives input focus (`FLAG_NOT_FOCUSABLE`), and it is removed by force-stopping the helper application. It exists only to produce a genuine overlay window so that the detector can be exercised against the same OS mechanism that real overlay malware relies on, rather than against a focus-loss proxy alone.

Vector 3 (C2 exfiltration). Malicious code executing in the banking application process constructs a network connection to an attacker-controlled host and transmits sensitive parameters. In this work this is simulated through a Frida-controlled trigger that causes the agent itself to construct the URL inside the protected process.

2.1.2 Assumptions

1. The research prototype operates on a rooted Android emulator with frida-server installed. This is an explicit research constraint; production deployment would use Frida Gadget or native integration.
2. The target application is InsecureBankv2 [29]. Results may not generalize to applications that use different network libraries (such as OkHttp, Retrofit or gRPC) or newer Android API levels without hook adaptation.
3. The ATS heuristic `deviceId < 0` has been validated only for the `adb input tap` simulation. ATS variants implemented through Accessibility Services may produce different `deviceId` values.
4. The Overlay scenario covers two complementary stimuli: a focus-loss proxy via `am start SETTINGS`, and a controlled real overlay APK (`OverlayTestApp`) that requests `SYSTEM_ALERT_WINDOW` and draws a `TYPE_APPLICATION_OVERLAY` window. The real overlay APK is a controlled, benign stimulus designed for this evaluation and does not represent the full diversity of real-world overlay malware.
5. The `TRUSTED_HOSTS` whitelist is defined statically and is suitable only for the controlled testbed.

2.2 System Architecture

2.2.1 Overview

The *RASP Dynamic Analyzer v2.0* consists of four runtime components, one controlled overlay stimulus application and two evaluation scripts interacting within an isolated research testbed. The four runtime components - the Frida agent, the controller, the attack simulator and the mock server - are shown in Figure 2.1. The overlay stimulus application (`OverlayTestApp`) is described in Section 2.5.5; the two evaluation scripts - a repeated-trial batch runner and a benign-baseline tester - orchestrate the runtime components and are described in Section 2.5.6. Figure 2.1 shows the data flow between the runtime components.

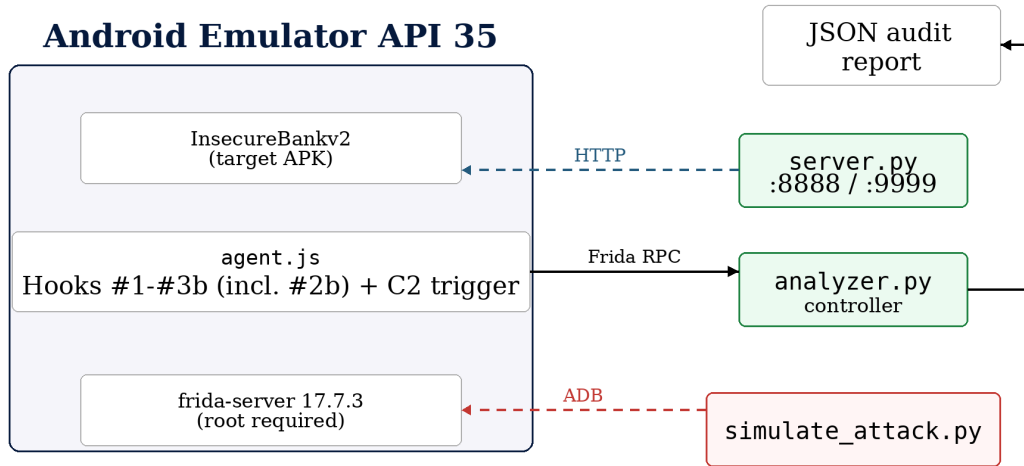


Figure 2.1: Architecture of *RASP Dynamic Analyzer v2.0* and the data flow between its components

The `agent.js` block in Figure 2.1 groups all detection rules under “Hooks #1- #3b”; Hook #2b is implemented inside the same `View.dispatchTouchEvent` interception point as Hook #1 and is therefore not drawn separately.

2.2.2 Testbed Configuration

Table 2.1 summarizes the configuration of the research testbed.

Table 2.1: Research testbed configuration

Component	Specification
Host OS	Linux CachyOS (Arch-based)
Emulator	Android Studio AVD, Pixel 6 profile, API 35, Google APIs x86_64 image
Root access	Via <code>adb root</code> ; the Google APIs image supports root, the Play Store image does not
Target app	InsecureBankv2 (<code>com.android.insecurebankv2</code>) [29]
Instrumentation	Frida 17.7.3 with frida-server android-x86_64
Python env	Python venv with frida-tools 14.6.0, colorama, Flask 3.x
ADB	Android Debug Bridge 35.0.2

The Google APIs image (without Play Store) was selected because it provides `adb root` capability in its developer build. The corresponding Play Store image enforces a production security policy that blocks root access. Irolla and Filiol [28] motivate the use of realistic emulator environments for dynamic analysis on Android.

2.3 Detection Logic and Decision Rules

Table 2.2 summarizes the detection logic implemented in each hook.

Table 2.2: Detection decision rules per hook

Hook	Intercepted call	Condition	Action	Level
#1	<code>View.dispatchTouchEvent</code>	<code>deviceId < 0</code>	Block + log	CRITICAL
#2	<code>Activity.onWindowFocusChanged</code>	<code>hasFocus = false</code> and IME not visible	Log	ALERT
#2b	<code>View.dispatchTouchEvent</code>	<code>MotionEvent.getFlags()</code> has <code>FLAG_WINDOW_IS_OBSCURED</code> or <code>FLAG_WINDOW_IS_PARTIALLY_OBSCURED</code>	Log	CRITICAL
#3a	<code>DefaultHttpClient.execute</code>	URL host not in trusted whitelist	Log	CRITICAL
#3a-ext	<code>OkHttpClient.newCall</code>	URL host not in trusted whitelist	Log	CRITICAL
#3b	<code>java.net.URL.\$init</code>	URL host not in trusted whitelist	Log	CRITICAL
any	legitimate network call	URL host in trusted whitelist	Log	WARNING

The prototype installs five interception points and six detection rules. Hooks #1 and #2b share the same `View.dispatchTouchEvent` interception point; the rest occupy one interception point each. The two overlay-related rules address complementary failure modes of UI deception. Hook #2, the focus-loss rule with an IME filter, is the correct

signal for overlays that take input focus (for example, by launching a foreground activity or by setting their overlay window as focusable). The IME filter prevents the rule from firing for ordinary soft-keyboard interactions. Hook #2 is, however, blind to overlays that deliberately avoid stealing focus: a real overlay window declared with the flag `FLAG_NOT_FOCUSABLE` does not trigger `onWindowFocusChanged` on the covered application. To close that gap, Hook #2b inspects the obscured-window flags carried by every `MotionEvent`. When another application's window is drawn on top of the banking application, the Android input subsystem marks touch events delivered to the covered window with `FLAG_WINDOW_IS_OBSCURED` or `FLAG_WINDOW_IS_PARTIALLY_OBSCURED` [17]; the presence of either flag on any single touch is the in-process evidence that an external window currently sits on top of the application. The two rules together cover both focus-stealing and non-focusable overlay variants. Hook #3a-ext targets `okhttp3.OkHttpClient.newCall` and extends network coverage to the OkHttp client library, which is widely used by modern Android applications. InsecureBankv2 does not bundle OkHttp, so this hook degrades silently at load time, reporting *not available*; it is wrapped in a guarded `try/catch` block for exactly this reason. The hook is retained as a guarded extension and documents the coverage boundary of the network layer; its evaluation on an application that actually bundles OkHttp is identified as future work.

2.4 Design Decisions and Alternatives Considered

Several implementation choices were made because the most direct solution was not always the most stable one in the testbed. Table 2.3 summarises the main alternatives considered and the rationale for each final choice.

Table 2.3: Design decisions and alternatives considered

Final decision	Alternative considered	Reason for the final choice
Frida CLI launched from <code>analyzer.py</code>	Direct use of the Frida Python API	Java Bridge initialization proved unstable through the Python API under Frida 17.x; the CLI initialized hooks reproducibly.
<code>View.dispatchTouchEvent</code> for ATS	Lower-level input interception	The selected hook exposes <code>MotionEvent</code> attributes close to the application logic and makes the audit explanation easier to verify.
IME-filtered focus-loss rule	Raw focus-loss alerting	Raw focus-loss detection produced an obvious false-positive risk because legitimate keyboard interaction also affects focus-related state.
File-based IPC for C2 trigger	Opening a second Frida session	A second Frida session conflicted with the active instrumentation workflow; a file trigger allowed the existing agent to execute the controlled C2-like request.
Guarded <code>okhttp3</code> hook	Hooking only the legacy HTTP client	A guarded <code>OkHttp</code> hook documents the network coverage boundary and stays inert when the library is absent, instead of silently ignoring modern HTTP clients.
JSON audit file	Terminal-only output	Terminal output is useful during testing, but a structured audit file is easier to inspect, archive and map to SIEM fields.

These decisions are part of the author’s engineering contribution. They also explain why the prototype is intentionally pragmatic: the controller is less elegant than a pure API-based implementation, but it is reproducible on the chosen emulator; the Overlay logic is not a complete malware detector, but it removes an obvious source of false positives; and the C2 trigger is synthetic, but it validates process-level URL interception without modifying the target APK.

2.5 Component Implementation

2.5.1 `agent.js`: the RASP Agent

`agent.js` is the core component of the prototype. It is loaded by Frida into the JVM of the target process. The entry point uses `Script.nextTick` to ensure correct Java Bridge initialization before the hooks are installed.

```

1 Script.nextTick(function() {
2   Java.perform(function() {
3     hookATS();           // Hook #1 + #2b (real overlay via obscured
                           flag)

```

```

4   hookOverlay();           // Hook #2
5   hookNetwork();           // Hooks #3a, #3a-ext, #3b
6   startC2Trigger();        // C2 file-trigger poll
7   });
8   });

```

Figure 2.2: agent.js entry point

Hook #1 - ATS detection (key snippet).

```

1 View.dispatchTouchEvent.implementation = function(ev) {
2   if (ev.getDeviceId() < 0) {                               // programmatic input
3     send({ type: "critical",
4           threat: "ATS Bot Activity",
5           message: "BLOCKED DeviceID: " + ev.getDeviceId() });
6     return true;                                           // consume event
7   }
8   return this.dispatchTouchEvent(ev);                      // pass through
9 };

```

Figure 2.3: ATS detection heuristic in Hook #1

In the tested environment, events from `adb input tap` consistently produced `deviceId = -1`. The hook consumes the event (returns `true`) before it reaches the application’s touch handling logic [17].

Hook #2 - Overlay-proxy detection (key snippet).

```

1 Activity.onWindowFocusChanged.implementation = function(hasFocus) {
2   if (!hasFocus) {
3     var imeVisible = this.getWindow().getDecorView()
4                       .getRootWindowInsets()
5                       .isVisible(Java.use("android.view.WindowInsets$Type").ime());
6     if (!imeVisible) {
7       send({ type: "alert", threat: "Overlay-proxy",
8             message: "Window focus lost (keyboard not open)" });
9     }
10  }
11  this.onWindowFocusChanged(hasFocus);
12 };

```

Figure 2.4: Overlay-proxy detection with IME filter in Hook #2

The `WindowInsets.Type.ime()` check distinguishes keyboard-related focus loss (normal behavior) from focus loss caused by other windows (potentially suspicious). This filter eliminates false positives generated by ordinary soft-keyboard interactions [30].

Hook #2b - Real overlay detection via obscured-touch flag (key snippet). A real `TYPE_APPLICATION_OVERLAY` window declared with `FLAG_NOT_FOCUSABLE` does not capture window focus and is therefore invisible to Hook #2. The corresponding in-process signal is delivered by the input subsystem: every touch event reaching a covered window carries one of two flags. Hook #2b reads these flags directly from the `MotionEvent` that Hook #1 already intercepts.

```

1 // MotionEvent flag bits (android.view.MotionEvent)
2 var FLAG_OBSCURED = 0x1 | 0x2; // FULL | PARTIAL
3
4 View.dispatchTouchEvent.implementation = function (ev) {
5   if (!overlayAlertFired && (ev.getFlags() & FLAG_OBSCURED) !== 0) {
6     overlayAlertFired = true;
7     send({ type: "critical", threat: "Overlay Attack",
8           message: "Touch input obscured by overlay window "
9                 + "(FLAG_WINDOW_IS_OBSCURED).
10              TYPE_APPLICATION_OVERLAY "
11                 + "window detected over banking UI." });
12   }
13 // ... ATS rule continues here (Hook #1)
14 };

```

Figure 2.5: Real overlay detection through obscured-touch flags in Hook #2b

A latch (`overlayAlertFired`) suppresses duplicate alerts so that a single overlay session emits one CRITICAL event rather than one per delivered touch. The two overlay hooks are complementary: Hook #2 catches focus-stealing variants, Hook #2b catches non-focusable variants such as the `OverlayTestApp` used in the evaluation (see Chapter 3).

Hooks #3a and #3b - network whitelist (key logic).

```

1 var TRUSTED = ["10.0.2.2:8888", "localhost", "127.0.0.1"];
2
3 function isTrusted(url) {
4   for (var i = 0; i < TRUSTED.length; i++)
5     if (url.indexOf(TRUSTED[i]) !== -1) return true;
6   return false;
7 }
8
9 // Used in both Hook #3a (DefaultHttpClient) and Hook #3b (URL.$init):
10 if (isTrusted(url))
11   send({ type: "warning", threat: "Network", message: url });
12 else

```

```

13     send({ type: "critical", threat: "C2 Exfiltration",
14           message: "SUSPICIOUS: " + url });

```

Figure 2.6: TRUSTED_HOSTS whitelist and classification logic

Hook #3b on `URL.$init` provides a deeper interception point for Java URL-based connections: it can detect network destinations constructed through `java.net.URL` before the connection is established. Hook #3a-ext additionally targets `okhttp3.OkHttpClient.newCall`; it is wrapped in a guarded `try/catch` block and stays inert when the `OkHttp` library is absent, as is the case for `InsecureBankv2`. Together the hooks still do not guarantee coverage for all Android networking stacks: `Retrofit`, `WebView` and `gRPC` require further hooks. The C2 simulation is triggered through a file-based IPC mechanism; selected implementation excerpts are given in Appendix A.

2.5.2 analyzer.py: the Controller

`analyzer.py` launches `Frida` as a subprocess, parses its output, displays events with color-coded severity and generates the JSON audit at the end of the session. The subprocess-based approach was required because the `Frida 17.x Java Bridge` did not initialize correctly via the Python API when connecting over USB to `frida-server`. The relevant controller behavior is summarized in Appendix A without reproducing the complete source file.

2.5.3 simulate_attack.py: the Attack Simulator

The simulator executes the attack scenarios sequentially through `ADB`. It supports four named scenarios (`ats`, `overlay_proxy`, `real_overlay` and `c2`) and a default run that executes the legacy three-vector sequence; `overlay` is kept as a backward-compatible alias for `overlay_proxy`. It uses `adb shell input tap` for `ATS`, `adb shell am start` for both the focus-loss proxy and the launch of the controlled overlay APK, and `adb shell touch` for the C2 file trigger. Only the essential simulator excerpt is included in Appendix A.

2.5.4 server.py: the Dual-Port Mock Server

A `Flask` application runs two HTTP servers on separate threads: port 8888 serves as the legitimate bank API, and port 9999 serves as the attacker's C2 endpoint. This separation

enables verification that the agent correctly distinguishes requests to the bank from requests to the attacker host. The mock server is described in the main text; the complete server source is omitted from the appendix.

2.5.5 OverlayTestApp: the Controlled Overlay Stimulus

`OverlayTestApp` (package `kz.aitu.overlaytest`) is a small Android application written specifically for this evaluation in order to produce a real overlay window over the banking application. The manifest requests `SYSTEM_ALERT_WINDOW`; the main activity calls `Settings.canDrawOverlays` and, when the permission is granted, starts a foreground service. The service uses `WindowManager` to add a `TextView` of fixed content (“TEST OVERLAY ACTIVE”) with overlay type `TYPE_APPLICATION_OVERLAY` and flag `FLAG_NOT_FOCUSABLE`, positioned at the top-centre of the screen, and removes the view in `onDestroy`. The application is benign by construction: it does not read input or screen contents, it does not communicate over the network, and the non-focusable flag prevents the overlay from intercepting taps or keystrokes targeted at the banking application underneath. Its sole purpose is to provide a controlled real-overlay stimulus that exercises the OS mechanism real overlay malware uses, so that Hook #2b can be evaluated against an actual `TYPE_APPLICATION_OVERLAY` window rather than a focus-loss proxy.

2.5.6 `batch_run.py` and `benign_baseline.py`: the Evaluation Scripts

Two scripts automate the evaluation so that the experiment is reproducible rather than a one-off manual procedure. `batch_run.py` executes the full single-run sequence - launch application, start server, start analyzer, wait for hooks, run the attack simulator, terminate the analyzer, parse the audit report - a configurable number of times, and aggregates the per-run event counts into a detection-rate table. The scenario to be exercised on each run is selected by the `--scenario` flag (one of `all`, `ats`, `overlay_proxy`, `real_overlay`, `c2`; `overlay` is accepted as a backward-compatible alias for `overlay_proxy`), which makes it possible to measure each detector under a dedicated stimulus rather than only as part of the combined sequence. `benign_baseline.py` performs the complementary measurement: it exercises four legitimate user actions while the analyzer is active and records whether any `CRITICAL` alert is incorrectly raised. The two scripts together produce the repeated-trial detection statistics and the false-positive baseline that are reported in Chapter 3.

2.6 Summary of Chapter 2

The system design includes the following:

- An explicit threat model covering ATS, overlay behavior and C2-like exfiltration with stated testbed assumptions and scope limitations.
- A four-component runtime architecture with clear separation between the Frida agent, Python controller, attack simulator and dual-port mock server.
- A controlled OverlayTestApp stimulus and two evaluation scripts for repeated-trial attack measurement and benign-baseline testing.
- Five interception points implementing six deterministic detection rules based on deviceId, IME state, MotionEvent obscured-window flags and trusted-host classification.
- A JSON audit format and file-based C2 trigger mechanism that preserve interpretable evidence while avoiding conflicting Frida sessions.

- Five interception points implementing six deterministic detection rules based on `deviceId`, IME state, the `MotionEvent` obscured-window flags and a host whitelist, including a guarded `okhttp3` extension hook that documents the network coverage boundary.
- An interpretable JSON audit schema that follows the Interpretable Dynamic Analysis principle [8].
- A file-based IPC mechanism for C2 simulation that avoids multi-session Frida conflicts.

Chapter 3

Experiment and Evaluation

This chapter describes the experimental procedure and the evaluation of the developed prototype. It begins with the methodology of data collection required by the AITU guidelines, continues with the repeated-trial procedure and the observed results, and concludes with classification analysis, interpretation, limitations and identified future work. The evaluation in this chapter follows a repeated-trial protocol: the full attack scenario was executed 20 times by an automated batch runner, and a separate benign baseline was executed to measure false positives.

3.1 Methodology of Data Collection

This section answers four questions: what data was collected, which sources produced it, why these sources are appropriate for the research goal and how the data was used in the evaluation.

3.1.1 Data Sources and Collection Methods

Table 3.1 summarizes the data sources, the data collected from each and their role in the validation of the prototype.

Table 3.1: Data collection methodology

Data source	Data collected	Collection tool	Validation role
Frida RPC message stream	Hook events (type, threat, message, timestamp)	<code>analyzer.py</code> / Frida CLI	Primary evidence of detection
Per-run JSON audit files	Full session event log with metadata	<code>analyzer.py</code> (auto-generated)	Per-run detection evidence; compact schema in Appendix A
Batch summary JSON	Per-run event counts and aggregate detection rates	<code>batch_run.py</code>	Repeated-trial detection statistics
Benign baseline JSON	Per-action CRITICAL/ALERT/WARNING counts	<code>benign_baseline.py</code>	False-positive measurement
Flask server log (port 8888)	Legitimate HTTP requests	<code>server.py</code> stdout	Baseline for legitimate traffic
Flask server log (port 9999)	C2 exfiltration requests	<code>server.py</code> stdout	Independent confirmation that the C2 request reached the attacker endpoint
ADB input tap commands	Triggered ATS events (<code>deviceId = -1</code>)	<code>simulate_attack.py</code>	Controlled stimulus for the ATS hook
ADB <code>am start</code> commands	Focus-loss events in InsecureBankv2	<code>simulate_attack.py</code>	Controlled stimulus for the Overlay-proxy hook
File trigger (<code>c2_trigger</code>)	C2 simulation trigger on the device	<code>simulate_attack.py</code> + <code>agent.js</code>	Controlled in-process C2 stimulus
OverlayTestApp + <code>MotionEvent</code> obscured flags	<code>FLAG_WINDOW_IS_OBSCURED</code> events; Overlay Attack CRITICAL event	<code>agent.js</code> Hook #2b + per-run JSON audit	Controlled real-overlay detection against a real <code>TYPE_APPLICATION_OVERLAY</code>

3.1.2 Justification of Data Sources

Frida event messages are the primary source of evidence because they originate from hooks executing inside the target process. This provides direct behavioral observability that is unavailable to external monitoring [7, 28]. The per-run JSON audit files are the atomic experimental artifact: each one records a complete monitoring session. The batch summary JSON aggregates the per-run counts and is the basis for the repeated-trial statistics. Flask server logs provide independent confirmation that network requests reached their destinations and therefore serve as ground truth for the classification of network events. ADB commands provide reproducible, controlled stimuli with known parameters, enabling repeated execution of the same attack scenario.

3.1.3 Justification of InsecureBankv2 as Research Target

InsecureBankv2 [29] was selected because it (1) provides a functional banking login workflow with HTTP communication, (2) contains documented intentional vulnerabilities suitable for controlled attack simulation and (3) is simple enough to support transparent

instrumentation and reproducible experimentation. The application does not represent a production banking system.

3.2 Experimental Procedure

3.2.1 Single-Run Procedure

Each individual experimental run performs the following sequence, orchestrated automatically by the batch runner:

1. Remove any stale `c2_trigger` file and force-stop the target application.
2. Launch `InsecureBankv2` and confirm via `frida-ps -U` that the process is present.
3. Start the dual-port mock server (`server.py`).
4. Start the analyzer (`analyzer.py`); wait until the log contains `=== All hooks active ===`.
5. Run the attack simulator (`simulate_attack.py`): six programmatic taps, the Settings focus-loss scenario, and the C2 file trigger.
6. Wait five seconds so the 500 ms C2 polling loop fires.
7. Send `SIGINT` to the analyzer; the analyzer writes the audit file atomically and prints a machine-readable `REPORT_SAVED` path.
8. Wait for the analyzer process to exit, wait until the report file is stable and valid JSON, then parse that exact report path and record the per-vector event counts.

3.2.2 Repeated-Trial Protocol

To obtain reproducibility evidence rather than a single observation, the procedure above was repeated 20 times by `batch_run.py`. The batch runner records, for every run, the number of ATS, Overlay-proxy and C2 events found in the corresponding audit report. The final documented main experiment was conducted on 21 May 2026 after the report-capture harness was stabilized.

3.2.3 Benign Baseline

A separate tool, `benign_baseline.py`, exercises four legitimate user actions while the analyzer is active: keyboard open/close with text input, application background/foreground transition, a legitimate HTTP request to the trusted bank endpoint, and screen rotation. Its purpose is to measure false positives, that is, CRITICAL alerts raised in the absence of an attack.

3.2.4 Attack Scenario Parameters

Table 3.2 describes the parameters of each simulated attack scenario.

Table 3.2: Simulated attack scenario parameters

Scenario	Stimulus	Expected observable	Scope note
ATS bot	<code>6 × adb shell input tap</code>	<code>deviceId = -1;</code> events blocked	Simulates <code>adb</code> injection only; not Accessibility-based ATS
Overlay proxy	<code>adb shell am start android.settings.SETTINGS</code>	Focus lost; IME not active	Proxy for focus-stealing overlays
Controlled overlay APK	<code>launch kz.aitu.overlaytest (TYPE_APPLICATION_OVERLAY, FLAG_NOT_FOCUSABLE) + one tap</code>	<code>MotionEvent</code> carries <code>FLAG_WINDOW_IS_OBSCURED</code>	Real <code>SYSTEM_ALERT_WINDOW</code> stimulus; benign, no credential capture
C2 exfiltration	File trigger $\rightarrow$ <code>URL.\$init</code> on port 9999	Non-whitelisted URL intercepted	Controlled Frida-triggered simulation; not real malware injection
Legitimate login	Manual mouse click on Login button	<code>deviceId ≥ 0</code> ; no block	Physical mouse; manual follow-up check

3.3 Experimental Results

3.3.1 Hook Initialization

In the updated analyzer used for the controlled overlay experiment, the startup log confirmed the installation of all hooks, including Hook #2b. The original three-vector reference session was produced before the dedicated Hook #2b overlay extension. A representative startup log from the updated analyzer is shown below:

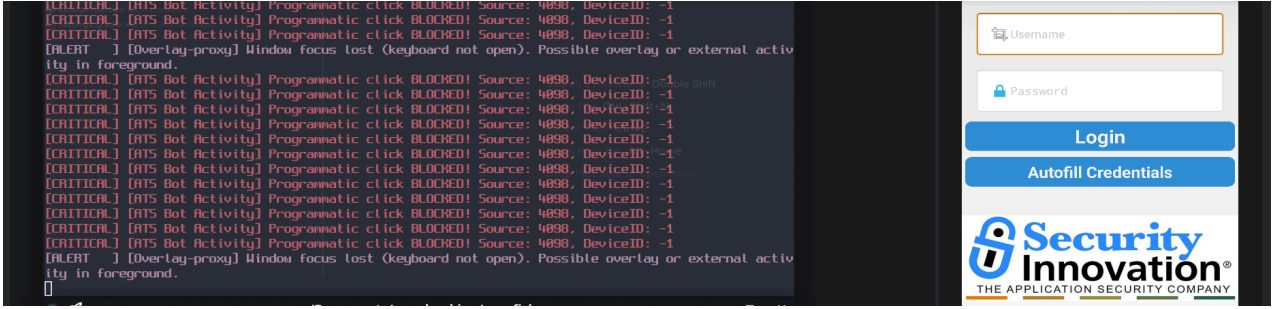


Figure 3.1: Runtime analyzer output during the combined attack scenario

The `okhttp3` hook reports *not available* because `InsecureBankv2` does not bundle the `OkHttp` library. This is the expected and correct behavior: the hook is wrapped in a guarded `try/catch` block and degrades silently. Its presence is a forward-compatibility measure discussed in Section 3.6.

3.3.2 Repeated-Trial Detection Results

The stabilized batch runner attempted 20 runs of the full three-vector scenario on 21 May 2026. All 20 runs produced valid JSON audit reports, and every valid report contained detections for ATS Bot Activity, Overlay-proxy behavior and C2 Exfiltration. The aggregate batch output was stored in `batch_results_20260521_205558.json`; its metadata records `total_runs = 20`, `valid_runs = 20` and `invalid_runs = 0`. The aggregate detection results are presented in Table 3.3.

Table 3.3: Repeated-trial detection results after harness stabilization (20 attempted runs, 20 valid)

Attack vector	Attempted runs	Valid reports	Detected	Detection rate
ATS Bot Activity	20	20	20	100%
Overlay-proxy	20	20	20	100%
C2 Exfiltration	20	20	20	100%

The key difference from the earlier measurement attempt is that the final runner no longer guesses the newest report file by modification time. Instead, `analyzer.py` writes the report through a temporary file followed by an atomic rename, prints `REPORT_SAVED=<path>`, and `batch_run.py` waits for process termination, file-size stabilization and successful JSON parsing. Therefore no run was lost because of report serialization timing. Within every valid run the per-vector event counts were stable:

12 ATS CRITICAL events, 1 Overlay-proxy ALERT and 1 C2 CRITICAL event per run. The twelve ATS events arise because the six injected taps generate `ACTION_DOWN` and `ACTION_UP` dispatch events, and all twelve potential dispatch events reached the hook in the final documented run.

3.3.3 Single-Run Detail and Latency

Run 20 of the stabilized experiment is used as the detailed reference session. It contains 19 events: 5 SYSTEM information messages, 12 ATS CRITICAL events, 1 Overlay-proxy ALERT and 1 C2 CRITICAL event.

Because the audit report records an ISO-8601 timestamp for every event, an approximate end-to-end latency can be computed directly from the report, measured from the `=== All hooks active ===` message. Table 3.4 reports these values for the reference session.

Table 3.4: End-to-end latency for the reference session (run 20), measured from hook activation

Milestone	Latency	Composition
Hooks active → first ATS event	1.36 s	Time until the simulator issues the first <code>adb input tap</code>
Hooks active → Overlay-proxy alert	4.82 s	Six taps at 0.5 s spacing, then the Settings activity launch
Hooks active → C2 event	7.56 s	Above, plus return to the app, the 500 ms trigger poll and the network call

These latency values are derived from the timestamps of one audit report and characterise that session only. They are dominated by the pacing of the attack simulator (deliberate `sleep` intervals between stimuli), not by hook overhead: each hook fires synchronously on its triggering event. A precise measurement of hook overhead in isolation was not instrumented and is identified as future work in Section 3.6.

3.3.4 Harness Stabilization and Report-Capture Verification

The final experiment uses a stabilized report-capture pipeline. In the earlier implementation, the analyzer was terminated with `SIGINT`, the batch runner waited a fixed interval, and then `latest_report()` selected the newest audit file by modification time. That

design could lose evidence if JSON serialization finished later than the fixed wait. The final implementation removes that race.

The stabilized analyzer accepts a deterministic session identifier, writes the audit report to a temporary file, flushes it to disk, and then renames it atomically to the final JSON filename. After the rename it prints a machine-readable `REPORT_SAVED` line. The batch runner parses that path, waits for the analyzer process to exit, waits until the file size is stable for consecutive checks, verifies that the file is non-empty and parses it as JSON before counting the run as valid.

This change is directly reflected in the final results: the batch summary records 20 attempted runs, 20 valid reports and 0 invalid reports. Therefore the final 100% result in Table 3.3 describes both the detector behavior and the stabilized evidence-capture pipeline.

3.3.5 Controlled Overlay APK Scenario

After the primary three-vector experiment, a dedicated scenario was added to evaluate Hook #2b against a real `TYPE_APPLICATION_OVERLAY` window rather than the focus-loss proxy. The stimulus is the `OverlayTestApp` helper application described in Section 2.5.5: it is launched from the simulator, draws a benign `FLAG_NOT_FOCUSABLE` overlay over the banking application, receives one injected tap so that the OS marks the touch with `FLAG_WINDOW_IS_OBSCURED`, and is then force-stopped to remove the window.

Table 3.5: Controlled overlay APK scenario - repeated-trial detection results

Attack vector	Attempted	Detected	Missed	Detection rate
Real overlay (Hook #2b)	20	20	0	100%

The figures in Table 3.5 come from the batch runner output (`batch_results_20260519_220928.json`, scenario `real_overlay`, 20 runs, `valid_runs = 20`, `RealOverlay.detected = 20`). The author independently inspected the per-run audit reports retained on disk from this batch. Eighteen audit files from the batch are available; two were not retained alongside the batch JSON. Of the 18 retained files, 17 strictly satisfy all four verification criteria simultaneously (Hook #2b initialization message in the `SYSTEM` events, an event with `level = "critical"`, `threat = "Overlay Attack"`, and a `details` field mentioning

FLAG_WINDOW_IS_OBSCURED or FLAG_WINDOW_IS_PARTIALLY_OBSCURED); the remaining file contains the required CRITICAL "Overlay Attack" event with the obscured-flag message but does not contain the explicit Hook #2b initialization line in its SYSTEM records.

Thus, 20/20 is the batch-level detection result, while 17/18 is the strict retained-artifact verification rate over the audit files that remained available for manual inspection. The two figures answer different questions and are reported separately: 20/20 reflects what the batch runner observed during execution; 17/18 reflects how many of those observations can still be independently re-verified from preserved evidence after the fact.

Counting rule. For the dedicated `real_overlay` scenario, only events with `threat = "Overlay Attack"` and `level = "critical"` are counted as Hook #2b detections. Overlay-proxy ALERT events caused by activity switching during the scenario are tracked separately and are explicitly *not* counted as real-overlay detections.

The `real_overlay` runs also report secondary events that are not the primary measurement but are worth stating plainly. Each run records four ATS CRITICAL events, generated by the single tap injected over the overlay together with the touch events produced when the simulator brings the banking activity back to the foreground; each run additionally records three Overlay-proxy ALERT events caused by the focus changes that surround launching and force-stopping the helper application. Neither set is part of the Hook #2b validation: the primary measurement is the per-run count of "Overlay Attack" CRITICAL events, which was 1 in every one of the 20 runs.

The 100% figure here describes the detector's response to one specific, controlled stimulus: an in-house benign overlay APK that exercises the exact OS mechanism (TYPE_APPLICATION_OVERLAY with FLAG_NOT_FOCUSABLE and a touch that triggers FLAG_WINDOW_IS_OBSCURED) on which Hook #2b depends. It is not a claim about coverage of all real-world overlay malware, which may use full-screen overlays, accessibility-driven UI hijacking, or other variants outside the scope of this stimulus.

3.3.6 Benign Baseline Results

The benign baseline exercised four legitimate user actions. The results are shown in Table 3.6.

Table 3.6: Benign baseline results (four legitimate scenarios)

Benign action	CRITICAL	ALERT	WARNING	Result
Keyboard open/-close (text input)	0	0	0	TN
App background → foreground	0	1	0	ALERT (known limitation)
Legitimate GET/login to bank:8888	0	0	0	TN
Screen rotation (portrait/landscape)	0	0	0	TN

No benign action produced a CRITICAL alert. Three of the four actions produced no alert of any kind. The application background/foreground transition produced one ALERT from the Overlay-proxy hook. This is an acknowledged limitation rather than a hidden defect, and it is discussed explicitly in Section 3.5.2 and Section 3.6: pressing HOME causes the activity to lose window focus while the soft keyboard is not visible, which is indistinguishable, at the level of the `onWindowFocusChanged` callback alone, from an external window appearing in the foreground.

The benign baseline is preliminary and covers only four representative actions. It is sufficient to check for obvious CRITICAL false positives in the evaluated workflow, but it is not a comprehensive usability or false-positive study. A broader baseline would extend the action set (for example, separate username and password field taps and typing, login button press, keyboard open/close as distinct events, screen rotation, and additional legitimate request types) and would distinguish between actions performed with a physical input device and synthetic `adb input tap` commands, since ADB-generated input itself belongs to the synthetic-input class that Hook #1 is designed to detect. A full benign ATS false-positive validation therefore requires physical mouse or touchscreen input on a device. The $TN = 4$ figure in Table 3.6 reflects the four scripted scenarios in this preliminary baseline and should not be read as a general claim about manual-input false-positive immunity.

3.3.7 C2 Traffic Verification

For each valid run, the C2 mock server on port 9999 logged the inbound exfiltration request, providing independent confirmation that the in-process URL construction produced an

actual network connection. A representative log line:

Figure 3.2: C2 server log line (port 9999)

```
[C2:9999] EXFILTRATION: /exfiltrate args={'login': 'jack',
    'pass': 'Jack@123', 'card': '4111111111111111'}
```

This confirms the end-to-end path: the agent constructed the URL, the `URL.$init` hook intercepted it before transmission, and the request nonetheless reached the simulated attacker endpoint, where it was recorded.

3.4 Classification Analysis

3.4.1 Confusion Matrix

The confusion matrix is built from the repeated-trial attack data and the benign baseline. For the attack side, each of the 20 valid runs contributes one detection outcome per vector. For the benign side, each of the four benign actions contributes one outcome. Table 3.7 presents the per-vector matrix.

Table 3.7: Confusion matrix per attack vector (20 valid attack runs, 4 benign actions)

Vector	TP	FN	FP	TN	Precision	Recall
ATS Bot Activity	20	0	0	4	1.00	1.00
Overlay-proxy	20	0	0*	3	1.00*	1.00
C2 Exfiltration	20	0	0	4	1.00	1.00

* For the Overlay-proxy vector, the background/foreground benign action produced one ALERT-level event. Because the threat model counts only CRITICAL events as positive detections, this ALERT is not a CRITICAL false positive; it is reported separately as a known ALERT-level limitation. Precision is therefore stated for CRITICAL-level decisions.

These results characterize the specific scenarios tested on Android API 35 with the described testbed. The 100% detection figure applies to the 20 stabilized runs that produced evidence. It is not a guarantee of detection under all real-world conditions, across all Android versions, or against all ATS, overlay or exfiltration implementations.

3.4.2 Evaluation Metrics

Three metrics were used. The Detection Rate (DR) is the ratio of correctly detected attacks to attacks for which a report exists. The False Positive Rate (FPR) is the ratio of benign actions incorrectly flagged at CRITICAL level to all benign actions. The end-to-end latency is estimated from audit-report timestamps as described in Section 3.3.3.

$$\text{DR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

Table 3.8: Evaluation metrics summary

Metric	Value	Note
Detection Rate (valid reports)	100% (20/20)	Per vector; no missed attacks in reports with evidence
Detection Rate (all attempts)	100% (20/20)	Final stabilized batch; 0 invalid reports
False Positive Rate (CRITICAL)	0% (0/4)	No benign action raised a CRITICAL alert
ALERT-level false positive	1 of 4 actions	Background/foreground transition; known limitation
Latency (hooks → ATS)	1.36 s	Reference session; simulator-pacing dominated
Latency (hooks → C2)	7.56 s	Reference session; simulator-pacing dominated

3.4.3 JSON Audit Report

Each session automatically produced a JSON audit file. The appendix now contains only a compact schema-level example, not a full audit dump. The structure is the following:

```

1 {
2   "report_meta": {
3     "generated_at": "2026-05-21T20:55:56.234072",
4     "tool": "RASP Dynamic Analyzer v2.0",
5     "target_app": "com.android.insecurebankv2",
6     "session_id": "run_020_20260521_205532",
7     "scenario": "all",
8     "total_events": 19
9   },
10  "events": [
11    { "timestamp": "...", "level": "critical",
12      "threat": "ATS Bot Activity",

```

```

13     "details": "Programmatic click BLOCKED! ... DeviceID: -1" },
14   { "timestamp": "...", "level": "alert",
15     "threat": "Overlay-proxy",
16     "details": "Window focus lost (keyboard not open). ..." },
17   { "timestamp": "...", "level": "critical",
18     "threat": "C2 Exfiltration",
19     "details": "SUSPICIOUS URL inside process: http
20   ://10.0.2.2:9999/..." }
21 ]

```

Figure 3.3: JSON audit report structure

This schema follows the Interpretable Dynamic Analysis principle [8]: every event contains a human-readable explanation together with the specific technical attribute that triggered the rule, that is, the `deviceId` value, the keyboard state or the suspicious URL. The format can be mapped to SIEM-compatible representations such as CEF or ECS for operational deployment.

3.5 Interpretation of Results

3.5.1 ATS Detection

The `deviceId < 0` heuristic differentiated programmatic input (`deviceId = -1`) from physical input across every valid run. The result supports the hypothesis that this attribute is a reliable ATS indicator for `adb`-based injection in the tested environment, and it is consistent with dynamic-analysis literature showing that runtime behavior reveals information unavailable to purely static inspection [7, 25]. The heuristic specifically covers ATS implemented through `adb input tap`; Accessibility-based variants may produce different `deviceId` values and are not covered [4, 22].

3.5.2 Overlay Proxy Detection

The IME filter, based on `WindowInsets.Type.ime()`, correctly suppressed alerts during keyboard interaction in the benign baseline. The Settings-launch stimulus reliably produced the ALERT event in every valid run. However, the benign background/foreground transition also produced one ALERT, because pressing HOME causes a focus loss without a visible IME, which the `onWindowFocusChanged` callback cannot distinguish from an

external window. This is a genuine limitation of a focus-loss proxy. It does not raise a CRITICAL alert, and the threat model therefore does not count it as a CRITICAL false positive, but it is reported transparently and a refinement is proposed in Section 3.6.

3.5.3 C2 Exfiltration Detection

The `URL.$init` hook intercepted the URL construction before the connection was established, and the C2 server log independently confirmed the request. This pre-transmission interception is the central advantage over external traffic monitoring: the sensitive parameters visible in the URL are captured in the audit before transport-layer encryption or external aggregation removes the application context. The observation is consistent with prior runtime-analysis work [6, 7, 26].

3.5.4 Reliability after Harness Stabilization

The final experiment no longer has a gap between detector output and persisted evidence. The stabilized batch runner produced 20 valid reports out of 20 attempted runs, and all three evaluated vectors were detected in every report. This is important because a runtime detector is only defensible if its evidence-capture pipeline is reliable. In the final pipeline, report identity is deterministic, report writing is atomic, and a run is counted only after the corresponding audit file is stable and valid JSON. The final result therefore supports the conclusion that both the detection rules and the measurement harness worked consistently under the tested scenario.

3.6 Limitations and Threats to Validity

3.6.1 Internal Validity

1. **Report-capture reliability.** The earlier fixed-wait report-capture race was eliminated in the final implementation by deterministic session identifiers, `REPORT_SAVED` parsing, process-exit waiting, file-size stabilization checks and atomic JSON writing. The final main experiment produced 20 valid reports out of 20 attempted runs.
2. **Single testbed.** All experiments ran on one Android emulator configuration. Results may differ on physical devices or other API levels.

3. **Controlled stimuli.** Attack scenarios were designed to trigger specific hooks, not to reproduce real malware.
4. **Latency not isolated.** The reported latency is dominated by simulator pacing; hook overhead in isolation was not instrumented.

3.6.2 External Validity

1. **Root requirement.** frida-server requires root; this is a research prototype, not a deployable product.
2. **ATS coverage.** The heuristic covers adb-based ATS; Accessibility-based ATS is not covered.
3. **Overlay coverage.** The initial version of this prototype used a focus-loss proxy. The updated implementation adds a controlled overlay APK (`OverlayTestApp`) and an obscured-touch flag detector (Hook #2b), which together cover both focus-stealing and non-focusable overlay variants. The result remains a controlled benign stimulus and does not represent the full diversity of real-world overlay malware (full-screen overlays, accessibility-driven UI hijacking, overlays carrying real fake-UI content, etc.).
4. **Network coverage.** `DefaultHttpClient` and `java.net.URL` are hooked, and an `okhttp3` hook is present but inactive because `InsecureBankv2` does not bundle `OkHttp`. `Retrofit`, `WebView` and `gRPC` are not covered.
5. **Single target application.** Results on `InsecureBankv2` may not transfer to production banking applications.

3.6.3 Threats to Validity

The principal threat to validity is the use of controlled, purpose-built attack stimuli rather than real malware samples. The benign baseline of four actions is also small. The results demonstrate that the detection logic functions for the modeled behavioral signatures; they cannot prove effectiveness against all attack implementations. This is the expected scope for a research prototype.

3.7 Future Work

1. **Instrument hook overhead.** Add direct measurement of per-hook latency, separated from simulator pacing.
2. **Test on physical devices.** Replicate the evaluation on a physical Android device, including a Frida Gadget deployment that does not require root.
3. **Broader ATS coverage.** Extend ATS detection to Accessibility-based automation via `AccessibilityService` callbacks, in addition to the current ADB-based path.
4. **Broader overlay validation.** Extend overlay validation beyond the controlled `OverlayTestApp` stimulus: full-screen overlays, fake-UI overlays, accessibility-driven UI hijacking, and a refinement of the focus-loss rule that distinguishes a system HOME press or other lifecycle transition from a malicious foreground activity.
5. **Activate the OkHttp path.** Validate the `okhttp3` hook on an application that actually bundles OkHttp, and add Retrofit, WebView and gRPC hooks.
6. **SIEM integration.** Export the JSON audit in CEF format for ingestion by Splunk, Elastic or IBM QRadar.

3.8 Summary of Chapter 3

- A structured data collection methodology was defined, covering ten data sources and their validation roles.
- The attack scenario was executed under a repeated-trial protocol of 20 runs; all 20 runs produced valid evidence after harness stabilization.
- Across the 20 valid runs, all three attack vectors were detected in every run: DR = 100% on valid reports and 100% across all attempts in the final stabilized batch.
- The benign baseline produced zero CRITICAL false positives; one ALERT-level event from a background/foreground transition is reported as a known limitation.
- A confusion matrix was constructed (per vector: TP = 20, FN = 0, FP = 0, TN = 3-4).

- A dedicated controlled overlay APK scenario was evaluated separately; the batch runner reported Hook #2b detection in 20 of 20 runs, with 17 of 18 retained per-run audit reports strictly satisfying all four verification criteria.
- The report-capture harness was stabilized through atomic JSON writing, deterministic report naming and explicit report-file validation; the final batch had 0 invalid reports.

Conclusion

This thesis developed and evaluated a runtime detection method for selected attack classes on Android mobile banking applications. The work was deliberately scoped as an engineering research prototype: it does not attempt to classify all Android malware families, but instead focuses on three behaviors that can be observed during an active application session - ADB-based touch injection, suspicious window-focus loss and C2-like construction of an untrusted network destination.

Achievement of the first task. The first research task was to conduct a literature review and narrow the mobile banking threat landscape to behaviors that can be reproduced on a controlled Android testbed. This was achieved through the selection of three scenarios: ATS, Overlay-proxy behavior and C2-like exfiltration. The literature review of 30 selected academic and official sources demonstrated that these behaviors correspond to documented Android banking and Android malware risks, while also confirming that the tested scenarios remain simplified models rather than complete malware implementations [1-5, 22, 23].

Achievement of the second task. The second task was to compare common defense approaches from the viewpoint of these scenarios. Static APK analysis is useful for finding insecure code patterns but cannot observe an externally generated `MotionEvent` or a runtime focus-loss callback. External network monitoring may observe traffic but cannot explain which in-process call produced a destination and cannot see input or window events. ML-based classifiers remain valuable for large-scale sample classification but are less suitable when the required output is an event-level explanation. This comparison justified the use of dynamic instrumentation and deterministic rules in the prototype.

Achievement of the third and fourth tasks. The third and fourth tasks concerned system design and implementation. The *RASP Dynamic An-*

alyzer v2.0 prototype was implemented with five interception points and six deterministic detection rules: `View.dispatchTouchEvent` (Hook #1 for ATS and Hook #2b for non-focusable overlay detection via `MotionEvent` obscured flags), `Activity.onWindowFocusChanged` (Hook #2 for focus-stealing overlays), `DefaultHttpClient.execute` (Hook #3a), `okhttp3.OkHttpClient.newCall` (Hook #3a-ext, guarded extension) and `java.net.URL.$init` (Hook #3b). The supporting workflow includes a JavaScript agent, a Python controller, an ADB-based scenario runner with named scenarios, a dual-port Flask server, automatic JSON audit generation, a repeated-trial batch runner with a `--scenario` switch, a benign-baseline tester, and a controlled `OverlayTestApp` stimulus that exercises Hook #2b.

The implementation also documents the main engineering constraints of the testbed, including Frida Java Bridge initialization, IME-aware focus-loss filtering, file-based C2 triggering, and guarded degradation of the OkHttp hook when the target application does not bundle the library.

Achievement of the fifth task. The fifth task was experimental evaluation under a repeated-trial protocol. After the report-capture harness was stabilized, the attack scenario was executed 20 times by an automated batch runner and all 20 runs produced valid audit reports. Across all 20 runs, every one of the three attack vectors was detected. The detection rate is therefore 100% over the 20 valid reports and 100% across all 20 attempted runs in the final stabilized batch. A separate benign baseline of four legitimate user actions produced zero CRITICAL false positives; one ALERT-level event from a background/foreground transition is documented as a known limitation. The latency figures (approximately 1.36 s to the first ATS event and 7.56 s to the C2 event in the reference session) are derived from audit-report timestamps and are dominated by the deliberate pacing of the attack simulator rather than by hook overhead.

Main contribution. The principal contribution of the thesis is the combination of runtime hooks with audit-oriented interpretation, evaluated under a repeated-trial protocol. Each alert is connected to a concrete observable attribute: `deviceId` for ADB-based ATS, IME visibility for focus-loss filtering and the destination URL for C2-like exfiltration. This makes the result more transparent than a binary classifier output. The generated JSON audit therefore serves not only as a log file but also as a structured explanation of why the prototype classified an event as suspicious [8].

Status of the hypothesis. The research hypothesis was supported within the

experimental scope. Deterministic runtime heuristics were sufficient to detect the three modeled scenarios in every run for which the stabilized harness captured evidence, without raising CRITICAL false positives on the tested legitimate interactions. At the same time, the limitations are significant and are stated plainly. The stabilized 20/20 report-capture result shows that the automated evaluation pipeline can preserve evidence reliably under the tested scenario. The prototype was evaluated on an emulator rather than physical devices; InsecureBankv2 is not a production banking application; the initial Overlay-proxy scenario validates the focus-loss rule, while the additional controlled `OverlayTestApp` scenario validates Hook #2b against a real `TYPE_APPLICATION_OVERLAY` condition - both remain controlled stimuli and do not represent the full diversity of real-world overlay malware; the C2 scenario is synthetic; and network coverage, although extended with a guarded `OkHttp` hook, has not been evaluated on an application that actually bundles `OkHttp`, and remains incomplete for `Retrofit`, `WebView` and `gRPC`.

Practical development path. A production-oriented version of the system would require instrumenting hook overhead in isolation, rootless deployment via Frida Gadget or native integration, broader hooks for modern Android networking libraries, validation on physical devices, broader overlay validation against full-screen overlays, accessibility-driven UI hijacking and more realistic fake-UI overlays, `OkHttp` validation on an application that actually bundles the library, and export of audit events to SIEM formats such as CEF or ECS. These steps are outside the scope of the current thesis, but they follow directly from the limitations identified during implementation and evaluation.

Bibliography

- [1] Kaspersky Lab.
Financial threat report 2024.
Securelist, 2024.
Available: <https://securelist.com/financial-threat-report-2024/115966/>.
- [2] European Union Agency for Cybersecurity.
ENISA threat landscape: Finance sector.
ENISA, reporting period January 2023- June 2024, 2025.
Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-finance-sector>.
- [3] Antonio Bianchi, Jacopo Corbetta, Luca Invernizzi, Yanick Fratantonio, Christopher Kruegel, and Giovanni Vigna.
What the app is that? Deception and countermeasures in the Android user interface.
In *2015 IEEE Symposium on Security and Privacy*, pages 931- 948, 2015.
- [4] Yanick Fratantonio, Chenxiong Qian, Simon P. Chung, and Wenke Lee.
Cloak and Dagger: From two permissions to complete control of the UI feedback loop.
In *2017 IEEE Symposium on Security and Privacy*, pages 1041- 1057, 2017.
- [5] Yuxuan Yan, Zhenhua Li, Qi Alfred Chen, Christo Wilson, Tianyin Xu, Ennan Zhai, Yong Li, and Yunhao Liu.
Understanding and detecting overlay-based Android malware at market scales.
In *Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pages 168- 179, 2019.
- [6] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth.

- TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones.
ACM Transactions on Computer Systems, 32(2):1- 29, 2014.
- [7] Lok Kwong Yan and Heng Yin.
DroidScope: Seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis.
In *Proceedings of the 21st USENIX Security Symposium*, pages 569- 584, 2012.
- [8] Arunab Singh, Maryam Tanha, Yashvi Girdhar, and Aaron Hunter.
Interpretable Android malware detection based on dynamic analysis.
In *Proceedings of the 10th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 195- 202, 2024.
- [9] Santanu Kumar Dash, Guillermo Suarez-Tangil, Salahuddin Khan, Kimberly Tam, Mansour Ahmadi, Johannes Kinder, and Lorenzo Cavallaro.
DroidScribe: Classifying Android malware based on runtime behavior.
In *2016 IEEE Security and Privacy Workshops (SPW)*, pages 252- 261, 2016.
- [10] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini.
MaMaDroid: Detecting Android malware by building Markov chains of behavioral models.
ACM Transactions on Privacy and Security, 22(2):1- 34, 2019.
- [11] Sufatrio, Darell J. J. Tan, Tong-Wei Chua, and Vrizlynn L. L. Thing.
Securing Android: A survey, taxonomy, and challenges.
ACM Computing Surveys, 47(4):1- 45, 2015.
- [12] OWASP Foundation.
OWASP mobile application security verification standard (MASVS).
Version 2.1.0, 2024.
Available: <https://mas.owasp.org/MASVS/>.
- [13] Gema Howell, Joshua M. Franklin, Vincent Sritapan, Murugiah Souppaya, and Karen Scarfone.
Guidelines for managing the security of mobile devices in the enterprise.
Technical Report NIST Special Publication 800-124 Rev. 2, National Institute of Standards and Technology, 2023.

- [14] Michael Ogata, Jeffrey Mell, Jeffrey Voas, Vincent Sritapan, and Stephen Quirolgico. Vetting the security of mobile applications. Technical Report NIST Special Publication 800-163 Rev. 1, National Institute of Standards and Technology, 2019.
- [15] OWASP Foundation. OWASP mobile application security testing guide (MASTG), 2024. Available: <https://mas.owasp.org/MASTG/>.
- [16] Frida Project. Frida: A world-class dynamic instrumentation toolkit, 2024. Available: <https://frida.re/docs/>.
- [17] Google LLC. MotionEvent: Android developers api reference, 2024. Available: <https://developer.android.com/reference/android/view/MotionEvent>.
- [18] Kimberly Tam, Ali Feizollah, Nor Badrul Anuar, Rosli Salleh, and Lorenzo Cavallaro. The evolution of Android malware and Android analysis techniques. *ACM Computing Surveys*, 49(4):1- 41, 2017.
- [19] Sen Chen, Ting Su, Lingling Fan, Guozhu Meng, Minhui Xue, Yang Liu, and Lihua Xu. Are mobile banking apps secure? What can be improved? In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 797- 802, 2018.
- [20] Parvez Faruki, Ammar Bharmal, Vijay Laxmi, Vijay Ganmoor, Manoj Singh Gaur, Mauro Conti, and Muttukrishnan Rajarajan. Android security: A survey of issues, malware penetration, and defenses. *IEEE Communications Surveys & Tutorials*, 17(2):998- 1022, 2015.
- [21] Loïc D. Tsobdjou, Samuel Pierre, and Alejandro Quintero. A framework for security assessment of Android mobile banking applications. *Computer Networks and Communications*, 2(1):49- 61, 2024.
- [22] Anatoli Kalysch, Davide Bove, and Tilo Müller. How Android’s UI security is undermined by accessibility.

- In *Proceedings of the 2nd Reversing and Offensive-Oriented Trends Symposium (ROOTS)*, pages 1- 10, 2018.
- [23] Yajin Zhou and Xuxian Jiang.
Dissecting Android malware: Characterization and evolution.
In *2012 IEEE Symposium on Security and Privacy*, pages 95- 109, 2012.
- [24] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Outeau, and Patrick McDaniel.
FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps.
In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 259- 269, 2014.
- [25] Mohammed K. Alzaylaee, Suleiman Y. Yerima, and Sakir Sezer.
Improving dynamic analysis of Android apps using hybrid test input generation.
arXiv:1705.06691, 2017.
Available: <https://arxiv.org/abs/1705.06691>.
- [26] Iker Burguera, Urko Zurutuza, and Simin Nadjm-Tehrani.
Crowdroid: Behavior-based malware detection system for Android.
In *Proceedings of the 1st ACM Workshop on Security and Privacy in Smartphones and Mobile Devices (SPSM)*, pages 15- 26, 2011.
- [27] Vaibhav Rastogi, Yan Chen, and William Enck.
AppsPlayground: Automatic security analysis of smartphone applications.
In *Proceedings of the Third ACM Conference on Data and Application Security and Privacy (CODASPY)*, pages 209- 220, 2013.
- [28] Paul Irolla and Eric Filiol.
Glassbox: Dynamic analysis platform for malware Android applications on real devices.
In *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*, pages 610- 621, 2017.
- [29] Dinesh Shetty.
Android-InsecureBankv2: Intentionally vulnerable Android application for security researchers.
GitHub repository, 2014.

Available: <https://github.com/dineshshetty/Android-InsecureBankv2>.

[30] Google LLC.

WindowInsets.Type: Android developers api reference, 2024.

Available: <https://developer.android.com/reference/android/view/WindowInsets.Type>.

Appendix A

Runtime Detection Rules in agent.js

The core of the prototype is the Frida agent. The following excerpt shows the three decision points that directly support the experimental claims: synthetic touch detection through `deviceId < 0`, real-overlay detection through `FLAG_WINDOW_IS_OBSCURED`, focus-loss filtering through IME visibility, and C2 detection through `java.net.URL` construction. Helper declarations, imports, logging decoration and duplicated boilerplate are omitted.

```
1 var TRUSTED = ["10.0.2.2:8888", "localhost", "127.0.0.1"];
2 function isTrusted(url) {
3   return TRUSTED.some(function (h) { return url.indexOf(h) !== -1; });
4 }
5
6 // Hook #1 + #2b: ADB touch injection and non-focusable overlay
7 View.dispatchTouchEvent.implementation = function (ev) {
8   if ((ev.getFlags() & 0x1) !== 0) {           // FLAG_WINDOW_IS_OBSCURED
9     send({ type: "critical", threat: "Overlay Attack",
10       message: "TYPE_APPLICATION_OVERLAY over banking UI" });
11   }
12   if (ev.getDeviceId() < 0) {                 // ADB-injected MotionEvent
13     send({ type: "critical", threat: "ATS Bot Activity",
14       message: "Programmatic click blocked; DeviceID: " + ev.
15       deviceId() });
16     return true;
17   }
18   return this.dispatchTouchEvent(ev);
19 };
20
21 // Hook #2: focus loss, filtered by soft-keyboard visibility
22 Activity.onWindowFocusChanged.implementation = function (hasFocus) {
23   var imeVisible = this.getWindow().getDecorView()
```

```

23     .getRootWindowInsets().isVisible(WindowInsetsType.ime());
24     if (!hasFocus && !imeVisible) {
25         send({ type: "alert", threat: "Overlay-proxy",
26             message: "Window focus lost; keyboard not open" });
27     }
28     this.onWindowFocusChanged(hasFocus);
29 };
30
31 // Hook #3b: C2-like destination construction inside the process
32 URL.$init.overload("java.lang.String").implementation = function (url)
33     {
34     if (!isTrusted(url)) {
35         send({ type: "critical", threat: "C2 Exfiltration",
36             message: "Suspicious URL inside process: " + url });
37     }
38     this.$init(url);
39 };

```

Figure 1: Essential runtime hooks in `agent.js`

Controlled Overlay and C2 Stimuli

The controlled overlay experiment uses a benign helper application. The relevant property is not the full Android project, but the window type and flags: the helper draws a `TYPE_APPLICATION_OVERLAY` window above the banking UI while remaining non-focusable and non-touchable. This is enough for Android to mark delivered touch events as obscured, which is the signal read by Hook #2b.

```

1 WindowManager.LayoutParams lp = new WindowManager.LayoutParams(
2     MATCH_PARENT,
3     MATCH_PARENT,
4     TYPE_APPLICATION_OVERLAY,
5     FLAG_NOT_FOCUSABLE | FLAG_NOT_TOUCHABLE,
6     PixelFormat.TRANSLUCENT
7 );
8 wm.addView(overlayView, lp);

```

Figure 2: Essential `OverlayTestApp` window parameters

The simulator needs only a short sequence to exercise this condition: bring the bank to the foreground, start the overlay helper, return the bank to the foreground while the

overlay service remains visible, and inject one tap. The C2 scenario is triggered through a sentinel file on the device; the agent then constructs the suspicious URL inside the protected process.

```

1 def run_real_overlay():
2     adb("am", "force-stop", "kz.aitu.overlaytest")
3     adb("am", "start", "-n", "com.android.insecurebankv2/.LoginActivity")
4     adb("am", "start", "-n", "kz.aitu.overlaytest/.MainActivity")
5     time.sleep(3)
6     adb("am", "start", "-n", "com.android.insecurebankv2/.LoginActivity")
7     adb("input", "tap", "540", "800")          # MotionEvent becomes
obscured
8     adb("am", "force-stop", "kz.aitu.overlaytest")
9
10 def run_c2():
11     adb("touch", "/data/local/tmp/c2_trigger")

```

Figure 3: Minimal simulator logic for overlay and C2 stimuli

Report Capture and Counting Rule

The analyzer stores every Frida event as a JSON object with timestamp, severity level, threat name and details. The batch runner then counts detections by threat name and severity. This is the only counting logic required to reproduce the reported metrics; process startup, cleanup, CSV export and other orchestration code are omitted.

```

1 def parse_report(path):
2     events = json.loads(path.read_text())["events"]
3     return {
4         "ats": sum(1 for e in events
5                     if e["threat"] == "ATS Bot Activity" and e["level"]
6                     == "critical"),
7         "overlay_proxy": sum(1 for e in events
8                               if e["threat"] == "Overlay-proxy" and e["level"] ==
9                               "alert"),
10        "real_overlay": sum(1 for e in events
11                             if e["threat"] == "Overlay Attack" and e["level"] ==
12                             "critical"),
13        "c2": sum(1 for e in events

```

```
11         if e["threat"] == "C2 Exfiltration" and e["level"]  
12         == "critical"),  
        }
```

Figure 4: Essential event counting rule used by the batch runner

A compact audit-schema example is shown below. It demonstrates the structure used by the experiments without dumping complete per-run reports into the appendix.

```
1 {  
2   "report_meta": {  
3     "tool": "RASP Dynamic Analyzer v2.0",  
4     "target_app": "com.android.insecurebankv2",  
5     "scenario": "all",  
6     "total_events": 19  
7   },  
8   "events": [  
9     { "level": "critical", "threat": "ATS Bot Activity",  
10      "details": "Programmatic click blocked; DeviceID: -1" },  
11     { "level": "alert", "threat": "Overlay-proxy",  
12      "details": "Window focus lost; keyboard not open" },  
13     { "level": "critical", "threat": "C2 Exfiltration",  
14      "details": "Suspicious URL inside process: http  
15      ://10.0.2.2:9999/..." }  
16   ]  
}
```

Figure 5: Compact JSON audit schema example